

1. Что называется системой контроля версий?

Система контроля версий (Version Control System, VCS) — это программная система, которая регистрирует изменения файлов и позволяет получать ранее сохранённые состояния этих файлов или всего проекта. В контролируемую историю включаются не все промежуточные действия, а выбранные завершённые изменения: разработчик сам определяет, когда текущее состояние следует сохранить как значимый этап работы.

2. Чем контроль версий отличается от автоматического сохранения?

Автоматическое сохранение в текстовом редакторе предотвращает потерю последних изменений, но обычно не формирует содержательную историю разработки. Оно не сообщает, какие файлы были изменены в рамках одной задачи, почему было принято решение и каким участником создано изменение. Контроль версий, напротив, фиксирует осмысленные завершённые состояния с метаданными: автором, датой, сообщением и связью с предыдущим состоянием.

3. Почему контроль версий не заменяет резервное копирование?

Это разные механизмы, которые дополняют друг друга. Резервное копирование предназначено прежде всего для восстановления данных после повреждения оборудования, удаления файлов, отказа накопителя или другого инцидента. Контроль версий предназначен для управления последовательностью содержательных изменений. Репозиторий тоже необходимо включать в систему резервного копирования, особенно если он существует только на одном компьютере

4. Какие задачи решает сохранение истории проекта?

История отвечает не только на вопрос «что находится в проекте сейчас», но и на вопросы «как проект пришёл к этому состоянию» и «какие решения принимались ранее». Качественная история полезна при сопровождении программного обеспечения: разработчик может обнаружить, что строка кода, которая сначала выглядит лишней, была добавлена для обработки редкого граничного случая. История особенно важна в проектах, которые развиваются много лет: участники команды меняются, а репозиторий сохраняет техническую память проекта.

5. Что называется версией и ревизией?

Слово «версия» используется в двух близких значениях. В первом значении версия — любое зафиксированное состояние; в этом случае каждый коммит образует новую версию проекта. Во втором значении версия — выпуск программного продукта, обозначенный номером, например 1.0 или 2.4.1. Один выпуск может соответствовать определённому коммиту, отмеченному специальным тегом. Ревизия — идентифицируемое состояние проекта в истории контроля версий. В централизованных системах ревизии часто имеют последовательные номера, а в Git состояния обычно обозначаются идентификаторами коммитов, именами веток, тегами или относительными ссылками.

6. Что представляет собой набор изменений?

Набор изменений — это группа связанных изменений, рассматриваемых как единое логическое целое. В Git таким зафиксированным набором обычно является коммит. Хороший набор изменений должен обладать содержательным единством: например, исправление ошибки авторизации не следует без необходимости объединять с изменением оформления интерфейса и переименованием десятков файлов. Независимые изменения лучше фиксировать отдельно.

7. Почему коммит рассматривается как снимок проекта?

Концептуально Git рассматривает коммит как снимок проекта, а не просто как последовательность команд редактирования. Если файл не изменился, Git не должен хранить ещё одну независимую полную копию его содержимого: объекты могут повторно использоваться по ссылкам. Для пользователя важно другое: каждый коммит позволяет обратиться к согласованному состоянию всего отслеживаемого проекта.

8. Чем локальная система отличается от локального репозитория Git?

В локальной системе история хранится на компьютере одного пользователя, система фиксирует версии файлов в локальной базе данных. Совместная работа организуется плохо, повреждение накопителя может привести к утрате истории. Локальный репозиторий Git — это хранилище истории, расположенное на компьютере пользователя, но Git при этом остаётся распределённой системой, даже если в конкретный момент пользователь работает только с локальным репозиторием. У каждого разработчика сохраняется локальный репозиторий, в котором можно создавать коммиты и просматривать историю автономно.

9. Как организована централизованная модель?

В централизованной системе основная история находится на центральном сервере. Пользователь получает рабочую копию файлов, изменяет их и передаёт результаты на сервер. Центральный сервер хранит общую историю, предоставляет единый адрес проекта, проверяет права пользователей, принимает изменения, обеспечивает совместный доступ, может выполнять резервное копирование и административный контроль. К централизованным системам относятся CVS, Apache Subversion и централизованные конфигурации Perforce Helix Core.

10. Какие риски связаны с центральным сервером?

Сервер становится критическим элементом инфраструктуры. Если он временно недоступен, часть операций совместной работы может оказаться невозможной. Если история хранится только на этом сервере и отсутствуют резервные копии, отказ оборудования создаёт риск её утраты. Кроме того, многие операции требуют сетевого соединения, автономная фиксация изменений может быть ограничена, а полная история может отсутствовать на рабочих компьютерах.

11. Как устроена распределённая модель?

В распределённой системе каждый разработчик обычно получает не только рабочие файлы, но и собственную копию репозитория с историей. Локальные операции выполняются без постоянного обращения к центральному серверу. Удалённый сервер часто используется как общее место обмена, однако он не превращает систему в централизованную. У каждого разработчика сохраняется локальный репозиторий, в котором можно создавать коммиты и просматривать историю автономно.

12. Почему Git остаётся распределённой системой при использовании общего сервера?

Потому что наличие удалённого сервера не меняет архитектуру: у каждого разработчика есть полная копия репозитория с историей. Локальные операции — создание коммитов, просмотр истории, сравнение состояний, создание веток — выполняются без сети. Удалённый сервер служит лишь местом обмена, но не единственным хранилищем истории. Это принципиальное отличие от централизованной модели, где рабочая копия не обязательно содержит полную историю проекта.

13. Чем Git отличается от GitHub?

Git — распределённая система контроля версий, это программа и совокупность форматов и правил работы с репозиториями. Git выполняет локальные операции: создаёт репозиторий,

обнаруживает изменения файлов, подготавливает выбранные изменения, формирует коммиты, хранит историю, создаёт ветки, объединяет изменения, взаимодействует с другими репозиториями. Для локальной работы GitHub не требуется. GitHub — платформа для размещения Git-репозитория и организации совместной разработки. Помимо хранения репозитория, она предоставляет учётные записи и организации, управление доступом, Issues, Pull Requests, рецензирование кода, автоматизацию GitHub Actions, публикацию выпусков, веб-интерфейс и средства управления проектами.

14. Какие функции GitLab не относятся непосредственно к Git?

GitLab — платформа, поддерживающая хранение Git-репозитория, рецензирование изменений, управление задачами и автоматизацию жизненного цикла разработки. Аналог запроса на включение изменений в GitLab называется Merge Request. GitLab может использоваться как облачный сервис или развёртываться в инфраструктуре организации. Как и GitHub, он не заменяет Git, а предоставляет серверные и организационные возможности вокруг Git-репозитория. К функциям, не относящимся непосредственно к Git, можно отнести управление задачами, рецензирование, автоматизацию жизненного цикла, Merge Request как объект платформы.

15. Как проверить установленную версию Git?

После установки выполняется команда:

```
git --version
```

Пример:

```
git version 2.x.x
```

Конкретный номер зависит от установленной версии. Если оболочка сообщает, что команда git не найдена, возможны следующие причины: Git не установлен; каталог с исполняемым файлом не включён в переменную PATH; терминал был открыт до завершения установки; используется неподходящая среда; установка завершилась с ошибкой. Сначала следует закрыть терминал, открыть его заново и повторить проверку.

16. Как получить справку по команде git commit?

Git имеет встроенную справочную систему. Полную документацию по команде можно открыть одним из способов:

```
git help commit
```

или:

```
git commit --help
```

Краткая справка выводится командой:

```
git commit -h
```

При изучении Git справочная система должна использоваться как основной источник точного синтаксиса. Краткие таблицы команд полезны для запоминания, но не заменяют документацию.

17. Для чего предназначена команда git config?

Команда `git config` предназначена для чтения и изменения параметров Git. Перед созданием первого коммита необходимо указать имя и адрес электронной почты:

```
git config --global user.name "Ivan Petrov"
```

```
git config --global user.email "ivan.petrov@example.org"
```

Эти сведения будут включаться в создаваемые коммиты. Имя в конфигурации Git не обязательно совпадать с именем учётной записи операционной системы. Проверить значения можно командами:

```
git config --global user.name
```

```
git config --global user.email
```

Просмотр всех доступных настроек:

```
git config --list
```

Просмотр настроек вместе с указанием файлов, из которых они получены:

```
git config --list --show-origin
```

18. Чем отличаются уровни system, global и local?

Уровень system применяется ко всем пользователям компьютера:

```
git config --system параметр значение
```

Изменение системной конфигурации может требовать административных прав.

Уровень global относится к текущему пользователю операционной системы:

```
git config --global параметр значение
```

Глобальная настройка применяется ко всем репозиториям пользователя, если в конкретном репозитории не задано другое значение. На этом уровне обычно задают имя, адрес электронной почты, редактор, общие параметры отображения, имя начальной ветки.

Уровень local относится только к текущему репозиторию:

```
git config --local параметр значение
```

Для записи в локальную конфигурацию необходимо находиться внутри Git-репозитория. Параметр `--local` можно не указывать, поскольку при записи внутри репозитория локальный уровень является стандартным.

19. Какие настройки имеют более высокий приоритет?

Если один параметр определён на нескольких уровнях, более конкретное значение имеет приоритет:

```
system -> global -> local
```

Локальная настройка репозитория переопределяет глобальную, а глобальная — системную.

20. Что называется рабочим каталогом?

Рабочий каталог, или working tree, — видимая пользователю совокупность файлов проекта в выбранном состоянии. В рабочем каталоге разработчик создаёт файлы, редактирует код, удаляет и перемещает элементы, запускает программу, выполняет тесты, просматривает результаты. Изменение файла в редакторе изменяет только рабочий каталог. Оно не становится частью истории автоматически.

21. Какую функцию выполняет индекс?

Индекс, *staging area*, — промежуточная область подготовки следующего коммита. Индекс содержит сведения о том, какое состояние файлов должно попасть в следующий коммит. Команда ``git add`` копирует в индекс текущее содержимое выбранных файлов. Русские варианты перевода термина *staging area*: индекс; область подготовленных изменений; область подготовки; промежуточная область. Для новичка полезно помнить его смысл: это проект будущего коммита. Индекс позволяет формировать осмысленные коммиты, не включая в них все изменения одновременно.

22. Что хранится в локальном репозитории?

Локальный репозиторий — хранилище зафиксированной истории проекта. Команда ``git commit`` создаёт новый коммит на основе состояния индекса. Она не фиксирует автоматически всё, что находится в рабочем каталоге. Репозиторий содержит: объекты с содержимым; деревья каталогов; коммиты; ссылки; ветки; теги; настройки; служебные журналы и другие данные Git. Служебные данные репозитория находятся в каталоге ``.git``. Удаление ``.git`` не удалит обычные файлы проекта, но уничтожит локальную историю и превратит каталог в обычную папку, не управляемую Git.